

Banco de Dados Relacional — Conceito e Definição

Um **Banco de Dados Relacional (BDR)** é um tipo de banco de dados que organiza as informações em **tabelas** (também chamadas de *relações*) formadas por linhas e colunas. Essas tabelas podem se **relacionar entre si** por meio de **chaves** (primárias e estrangeiras), permitindo que os dados sejam integrados e consultados de forma eficiente.

Definição formal

Um **Banco de Dados Relacional** é um sistema de armazenamento de dados baseado no **modelo relacional**, proposto por **Edgar F. Codd** em 1970.

Nesse modelo:

- Os dados são representados como **conjuntos de relações (tabelas)**.
- Cada tabela representa uma entidade ou conceito do mundo real.
- As relações entre tabelas são estabelecidas por **chaves**.
- As operações de manipulação de dados são feitas utilizando **Linguagem SQL (Structured Query Language)**.

Estrutura básica de um Banco de Dados Relacional

Elemento	Definição
Tabela (Relação)	Conjunto organizado de dados <i>sobre um mesmo tema</i> .
Linha (Tupla)	Um registro da tabela — um item da entidade.
Coluna (Atributo)	Característica que descreve a entidade.
Chave Primária	Identificador único de cada registro.
Chave Estrangeira	Campo que estabelece relação com outra tabela.

Tabelas

Cliente

id_cliente	nome	cidade
1	Ana	SP
2	João	PR

Pedidos

id_pedido	data	id_cliente
100	2025-11-29	1
101	2025-11-29	2

A coluna **id_cliente** em *Pedidos* é uma **chave estrangeira** que referencia a tabela *Cliente*

Benefícios do Modelo Relacional

- ✓ Estrutura clara e organizada
- ✓ Facilidade de consulta com **SQL**
- ✓ Garantia de integridade dos dados
- ✓ Relações bem definidas
- ✓ Ampla utilização no mercado (MySQL, PostgreSQL, Oracle, SQL Server)

Banco de Dados Relacional vs. NoSQL

1. Estrutura dos Dados

Relacional (SQL)

- Dados organizados em **tabelas**, linhas e colunas.

- Estrutura **fixa** (schema rígido).
- Relacionamento por **chaves** (PK–FK).

NoSQL

- Estrutura **flexível**, podendo mudar a qualquer momento.
- Não usa apenas tabelas — existem diferentes tipos:
 - ✓ Documentos (JSON)
 - ✓ Chave–valor
 - ✓ Colunar
 - ✓ Grafos
- Cada registro pode ter campos diferentes.

2. Linguagem de Consulta

Relacional

- Usa **SQL**, padrão mundial, muito poderoso para consultas complexas.

NoSQL

- Muitas vezes **não usa SQL** (embora alguns suportem variações como o MongoDB Aggregation).
- Cada banco tem sua própria forma de consultar.

3. Modelo de Dados

Relacional

- Ideal quando existe **relacionamento forte** entre entidades.
- Depende de **normalização** para evitar redundância.
- Mais adequado para estruturas que não mudam muito.

NoSQL

- Pode **desnormalizar**: dados podem ser duplicados para facilitar leitura.
- Ideal para dados **heterogêneos**, sem formato fixo.
- Ótimo para evoluir o modelo rápido.

4. Escalabilidade

Relacional

- Escalabilidade vertical (scale up):
→ aumentar CPU, RAM, disco do servidor.
- Cluster é possível, mas mais complexo.

NoSQL

- Escalabilidade horizontal (scale out):
→ adicionar mais máquinas simples ao cluster.
- Criado para ambientes distribuídos.

5. Performance

Relacional

- Excelente para consultas complexas (JOINS, agregações).
- Menos eficaz quando há grande volume não estruturado.

NoSQL

- Muito rápido em leitura/escrita de dados **massivos**.
- Feito para Big Data, tempo real, e picos de carga.

6. Consistência vs. Disponibilidade (CAP Theorem)

Relacional

- Prioriza **consistência**.
Exemplo: transação ACID (tudo ou nada).

NoSQL

- Alguns priorizam **disponibilidade** e distribuição.
Muitos usam o modelo **BASE** (eventual consistency).

7. Exemplos de Banco

Relacional

- MySQL
- PostgreSQL
- Oracle
- SQL Server

NoSQL

- **Documentos** → MongoDB, CouchDB
- **Chave-valor** → Redis
- **Grafos** → Neo4j
- **Colunar** → Cassandra, HBase

Resumo Geral

Característica	Relacional (SQL)	NoSQL
Estrutura	Tabelas	Flexível (JSON, chave-valor, grafo...)
Schema	Rígido	Flexível

Escalabilidade	Vertical	Horizontal
Transações	ACID	BASE (geralmente)
Relações	Fortes (JOINS)	Fracas ou embutidas
Ideal para	Dados estruturados e consistência	Big Data, alta performance e flexibilidade

Quando usar cada um?

Use Relacional quando:

- há regras de negócio fortes
- existem muitos relacionamentos
- você precisa de **consistência transacional**
- o modelo de dados muda pouco

Use NoSQL quando:

- grande volume de dados
- dados sem estrutura fixa
- necessidade de **alta escalabilidade**
- sistemas distribuídos
- aplicações web e mobile que mudam rápido

Comparação: NoSQL (documento) vs Banco Relacional (tabelas)

Vamos usar o exemplo NoSQL abaixo como referência:

```
{
```

```
{
  "_id": "PED12345",
  "data_pedido": "2025-11-29T14:33:00",
  "cliente": {
    "id_cliente": 1,
    "nome": "Ana Souza",
    "email": "ana@gmail.com",
    "telefone": "1199999-1111",
    "endereco": {
      "rua": "Rua das Flores",
      "numero": 100,
      "cidade": "São Paulo",
      "uf": "SP"
    }
  },
  "itens": [
    {
      "produto_id": "P001",
      "descricao": "Milho 1kg",
      "quantidade": 2,
      "preco_unitario": 5.50,
      "subtotal": 11.00
    },
    {
      "produto_id": "P002",
      "descricao": "Feijão 1kg",
      "quantidade": 1,
      "preco_unitario": 8.90,
      "subtotal": 8.90
    }
  ],
  "total_pedido": 19.90,
  "status": "aguardando_pagamento"
}
```

Como ficaria o MESMO pedido em um Banco Relacional

No relacional, tudo precisa ser **separado em tabelas** e conectado por **chaves primárias e estrangeiras**. Os dados que no NoSQL ficam embutidos (“cliente”, “endereços”, “itens”) precisam ser NORMALIZADOS.

Tabelas Necessárias (Modelo Relacional)

Tabela: Cliente

id_cliente (PK)	nome	email	telefone
1	Ana Souza	ana@gmail.com	1199999-1111

Tabela: Endereco

id_endereco (PK)	id_cliente (FK)	rua	numero	cidade	uf
10	1	Rua das Flores	100	São Paulo	SP

No relacional, o endereço é separado do cliente.

Tabela: Pedido

id_pedido (PK)	data_pedido	id_cliente (FK)	total_pedido	status
PED12345	2025-11-29 14:33	1	19.90	aguardando_pagamento

Tabela: Produto *(normalmente existe separada)*

produto_id (PK)	descricao	preco_atual
P001	Milho 1kg	5.50
P002	Feijão 1kg	8.90

Tabela: ItensPedido

id_item (PK)	id_pedido (FK)	produto_id (FK)	quantidade	preco_unitario	subtotal
1	PED12345	P001	2	5.50	11.00
2	PED12345	P002	1	8.90	8.90

Resumo das diferenças

NoSQL (Documento único)

Tudo está dentro de **um único registro**:

- Dados do cliente
- Endereço
- Itens
- Totais
- Status

É como uma “ficha completa do pedido” em formato JSON.

Banco Relacional (várias tabelas)

O mesmo pedido vira:

- **Cliente** (tabela)
- **Endereço** (tabela)
- **Pedido** (tabela)
- **ItensPedido** (tabela)
- **Produto** (tabela)

Todos conectados por **PK–FK**.

Comparação Geral entre os Modelos

Aspecto	NoSQL (Documento)	Relacional (Tabelas)
Armazenamento	Tudo em um único documento JSON	Dados distribuídos em várias tabelas

Relacionament o	Embutido (dados do cliente dentro do pedido)	Propriedade das tabelas com PK–FK
Normalização	Baixa (dados duplicados)	Alta (evita redundância)
Leitura do pedido	Muito rápida (1 documento)	Requer JOINS (3–6 tabelas)
Atualizar cliente	Difícil (precisa atualizar vários documentos)	Fácil (1 linha na tabela Cliente)